Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Сивец Д.Ю.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.11.26

Москва, 2026

Постановка задачи

Вариант 2.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Требуется взять первую лабораторную работу и переделать её с использованием разделяемой памяти (shared memory) и отображения памяти (memory mapping). Объект разделяемой памяти должен быть именованным и находиться в оперативной памяти, для синхронизации чтения и записи используется семафор. Имена объектов не должны быть константными, чтобы разные экземпляры программ не мешали друг другу.

Вариант 2. Родительский процесс создаёт дочерний процесс. Первой строкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процессы должны быть представлены разными программами. Пользователь вводит команды вида «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- int shm_open(const char *name, int oflag, mode_t mode); – создаёт или открывает именованный объект разделяемой памяти.
- int ftruncate(int fd, off_t length); – задаёт размер объекта разделяемой памяти.
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); – отображает объект разделяемой памяти в адресное пространство процесса.
- int munmap(void *addr, size_t length); – удаляет отображение.
- int shm_unlink(const char *name); – удаляет имя объекта разделяемой памяти.
- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); – создаёт или открывает именованный семафор.
- int sem_wait(sem_t *sem); – уменьшает значение семафора или блокирует процесс, пока оно равно нулю.
- int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout); – то же, что sem_wait, но с ограничением времени ожидания.
- int sem_post(sem_t *sem); – увеличивает значение семафора и будит ожидающий процесс.
- int sem_close(sem_t *sem); – закрывает семафор в процессе.
- int sem_unlink(const char *name); – удаляет имя семафора.
- pid_t fork(void); – создаёт дочерний процесс.
- int execl(const char *path, const char *arg, ...); – заменяет образ процесса программой child.
- pid_t waitpid(pid_t pid, int *status, int options); – ожидает завершения дочернего процесса.
- ssize_t read(int fd, void *buf, size_t count); – читает ввод пользователя.

- ssize_t write(int fd, const void *buf, size_t count); – записывает результат в файл и сообщения об ошибках.
- int open(const char *pathname, int flags, mode_t mode); – создаёт файл для результата.
- int close(int fd); – закрывает файловый дескриптор.

Программа написана на языке Си и запускается в ОС Linux (WSL2, Ubuntu). Она состоит из двух программ – parent и child – и общего заголовочного файла common.h. Вместо канала pipe из первой лабораторной работы данные передаются через именованную разделяемую память POSIX, а синхронизация выполняется двумя именованными семафорами.

Разделяемая память. Родитель создаёт объект функцией shm_open с флагами O_CREAT | O_EXCL, задаёт ему размер структуры shm_t функцией ftruncate и отображает его в свою память функцией mmap с флагом MAP_SHARED. Структура shm_t содержит буфер на 4096 байт и поле len – количество байт данных в буфере. Объект находится в оперативной памяти (файловая система tmpfs /dev/shm), а не на диске. Дочерний процесс открывает тот же объект по имени и тоже отображает его в свою память, поэтому оба процесса работают с одними и теми же физическими страницами.

Уникальные имена. Имена объектов составляются из префикса и идентификатора родительского процесса, например /lab3_shm_515, /lab3_empty_515, /lab3_full_515. Поэтому у каждой запущенной пары программ свои объекты, а флаг O_EXCL не даёт подключиться к чужому объекту. Имена передаются дочернему процессу аргументами execl.

Синхронизация. Используются два семафора: empty («буфер свободен», начальное значение 1) и full («в буфере есть данные», начальное значение 0). Родитель читает очередную порцию ввода, ждёт семафор empty, копирует данные в разделяемую память и увеличивает full. Дочерний процесс ждёт full, обрабатывает данные и увеличивает empty. Так процессы обращаются к буферу строго по очереди и не читают недописанные данные. Конец ввода родитель сообщает порцией с len = 0.

Обработка данных. Ввод передаётся порциями до 4096 байт, поэтому число или строка может оказаться разделённой между двумя порциями. Дочерний процесс хранит недочитанное число и сумму текущей строки между порциями. На каждом символе перевода строки он записывает в файл сумму чисел строки с двумя знаками после запятой (функция write_sum из первой лабораторной работы). Так как условие можно понимать и как сумму каждой команды, и как сумму всех введённых чисел, после окончания ввода последней строкой файла записывается сумма всех чисел.

Обработка ошибок. Проверяется результат каждого системного вызова. Если дочерний процесс завершился раньше времени (например, не смог создать файл), семафор empty больше никто не увеличит. Поэтому родитель ждёт его функцией sem_timedwait с таймаутом 1 секунда и между попытками проверяет дочерний процесс функцией waitpid с флагом WNOHANG. После завершения работы и при любой ошибке родитель удаляет отображение и объекты (munmap, sem_close, shm_unlink, sem_unlink), чтобы они не остались в /dev/shm.

Код программы

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <unistd.h>

#define BUF_SIZE 4096

typedef struct {
    int len;
    char data[BUF_SIZE];
```

```
} shm_t;

static inline void put_err(const char* s) {
    int len = 0;
    while (s[len]) len++;
    if (write(2, s, len) < 0) return;
}

static inline void make_name(char* out, const char* prefix, long id) {
    int len = 0;
    while (prefix[len]) { out[len] = prefix[len]; len++; }
    char tmp[24]; int t = 0;
    do { tmp[t++] = (char)('0' + id % 10); id /= 10; } while (id > 0);
    while (t > 0) out[len++] = tmp[--t];
    out[len] = '\0';
}

#endif
```

parent.c

```
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <time.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include "common.h"

static char shm_name[64], empty_name[64], full_name[64];
static shm_t* shm = MAP_FAILED;
static sem_t* empty = SEM_FAILED;
static sem_t* full = SEM_FAILED;

static void cleanup(void) {
    if (shm != MAP_FAILED) munmap(shm, sizeof(shm_t));
    if (empty != SEM_FAILED) sem_close(empty);
    if (full != SEM_FAILED) sem_close(full);
    shm_unlink(shm_name);
    sem_unlink(empty_name);
    sem_unlink(full_name);
}

static int read_line(int fd, char* buf, int cap) {
    int len = 0;
    char c;
    while (len < cap - 1 && read(fd, &c, 1) == 1) {
        if (c == '\n') break;
        buf[len++] = c;
    }
    buf[len] = '\0';
    return len;
}

static int wait_empty(pid_t pid, int* status) {
    for (;;) {
        struct timespec ts;
        clock_gettime(CLOCK_REALTIME, &ts);
        ts.tv_sec += 1;
        if (sem_timedwait(empty, &ts) == 0) return 0;
        if (errno == EINTR) continue;
        if (errno != ETIMEDOUT) return -1;
        if (waitpid(pid, status, WNOHANG) == pid) return 1;
    }
}

int main() {
    char fname[256];
    if (read_line(0, fname, sizeof(fname)) <= 0) {
        put_err("parent: expected filename on first line\n");
        return 1;
    }
```

```
long id = getpid();
make_name(shm_name, "/lab3_shm_", id);
make_name(empty_name, "/lab3_empty_", id);
make_name(full_name, "/lab3_full_", id);
int fd = shm_open(shm_name, O_CREAT | O_EXCL | O_RDWR, 0600);
if (fd == -1) {
    put_err("parent: shm_open failed\n");
    return 1;
}
if (ftruncate(fd, sizeof(shm_t)) == -1) {
    put_err("parent: ftruncate failed\n");
    close(fd);
    cleanup();
    return 1;
}
shm = mmap(NULL, sizeof(shm_t), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
close(fd);
if (shm == MAP_FAILED) {
    put_err("parent: mmap failed\n");
    cleanup();
    return 1;
}
empty = sem_open(empty_name, O_CREAT | O_EXCL, 0600, 1);
full = sem_open(full_name, O_CREAT | O_EXCL, 0600, 0);
if (empty == SEM_FAILED || full == SEM_FAILED) {
    put_err("parent: sem_open failed\n");
    cleanup();
    return 1;
}
pid_t pid = fork();
if (pid == -1) {
    put_err("parent: fork failed\n");
    cleanup();
    return 1;
}
if (pid == 0) {
    execl("./child", "child", fname, shm_name, empty_name, full_name, (char *)NULL);
    put_err("parent: exec ./child failed\n");
    _exit(1);
}
char buf[BUF_SIZE];
int status = 0, exited = 0;
for (;;) {
    ssize_t n = read(0, buf, sizeof(buf));
    if (n < 0) {
        put_err("parent: read failed\n");
        n = 0;
    }
    int r = wait_empty(pid, &status);
    if (r == 1) {
        exited = 1;
        break;
    }
    if (r == -1) {
        put_err("parent: sem_wait failed\n");
        break;
    }
    for (ssize_t i = 0; i < n; i++) shm->data[i] = buf[i];
    shm->len = (int)n;
    sem_post(full);
    if (n == 0) break;
}
if (!exited && waitpid(pid, &status, 0) == -1) {
    put_err("parent: waitpid failed\n");
    cleanup();
    return 1;
}
cleanup();
if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
    if (code != 0) {
        put_err("parent: child reported error\n");
        return code;
```

```
        }
        return 0;
    }
    put_err("parent: child terminated abnormally\n");
    return 1;
}
```

child.c

```
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <stdlib.h>
#include <semaphore.h>
#include <sys/mman.h>
#include "common.h"

static void write_sum(int fd, float sum, int decimals) {
    if (decimals > 7) {
        decimals = 7;
    }
    char buf[64]; int len = 0;
    if (sum < 0) { buf[len++] = '-'; sum = -sum; }
    long whole = (long)sum;
    long scale = 1;
    for (int i = 0; i < decimals; i++) scale *= 10;
    long frac = (long)((float)(sum - (float)whole) * (float)scale + 0.5);
    if (frac == scale) { whole += 1; frac = 0; }
    char tmp[48]; int t = 0;
    if (whole == 0) tmp[t++] = '0';
    while (whole > 0) { tmp[t++] = (char)('0' + whole % 10); whole /= 10; }
    for (int i = t - 1; i >= 0; i--) buf[len++] = tmp[i];
    if (decimals > 0) {
        buf[len++] = '.';
        t = 0;
        for (int i = 0; i < decimals; i++) { tmp[t++] = (char)('0' + frac % 10); frac /= 10; }
        for (int i = t - 1; i >= 0; i--) buf[len++] = tmp[i];
    }
    buf[len++] = '\n';
    if (write(fd, buf, len) != len) put_err("child: write to file failed\n");
}

int main(int argc, char* argv[]) {
    if (argc < 5) {
        put_err("usage: child filename shm_name empty_name full_name\n");
        return 1;
    }
    int out = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
        put_err("child: cannot open file\n");
        return 1;
    }
    int fd = shm_open(argv[2], O_RDWR, 0);
    if (fd == -1) {
        put_err("child: shm_open failed\n");
        return 1;
    }
    shm_t* shm = mmap(NULL, sizeof(shm_t), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    close(fd);
    if (shm == MAP_FAILED) {
        put_err("child: mmap failed\n");
        return 1;
    }
    sem_t* empty = sem_open(argv[3], 0);
    sem_t* full = sem_open(argv[4], 0);
    if (empty == SEM_FAILED || full == SEM_FAILED) {
        put_err("child: sem_open failed\n");
        return 1;
    }
    float sum = 0.0f, total = 0.0f;
    int cnt = 0, rc = 0;
    char tok[64];
    int tlen = 0;
```

```
    for (;;) {
        if (sem_wait(full) == -1) {
            if (errno == EINTR) continue;
            put_err("child: sem_wait failed\n");
            rc = 1;
            break;
        }
        int len = shm->len;
        int n = len > 0 ? len : 1;
        for (int i = 0; i < n; i++) {
            char c = len > 0 ? shm->data[i] : '\n';
            if (c == ' ' || c == '\n' || c == '\t' || c == '\r') {
                if (tlen > 0) {
                    tok[tlen] = '\0';
                    float v = (float)atof(tok);
                    sum += v;
                    total += v;
                    cnt++;
                    tlen = 0;
                }
                if (c == '\n' && cnt > 0) {
                    write_sum(out, sum, 2);
                    sum = 0.0f;
                    cnt = 0;
                }
            } else if (tlen < 63) {
                tok[tlen++] = c;
            }
        }
        sem_post(empty);
        if (len == 0) break;
    }
    if (rc == 0) write_sum(out, total, 2);
    munmap(shm, sizeof(shm_t));
    sem_close(empty);
    sem_close(full);
    close(out);
    return rc;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(OS_lab3 C)

set(CMAKE_C_STANDARD 11)

find_package(Threads REQUIRED)

add_executable(parent parent.c)
add_executable(child child.c)
target_link_libraries(parent Threads::Threads)
target_link_libraries(child Threads::Threads)
```

Протокол работы программы

Сборка программы (WSL2, Ubuntu):

```
$ cmake -S . -B build
-- The C compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (0.8s)
-- Generating done (0.0s)
-- Build files have been written to: /tmp/lab3/build
$ cmake --build build
```

```
[ 25%] Building C object CMakeFiles/parent.dir/parent.c.o
[ 50%] Linking C executable parent
[ 50%] Built target parent
[ 75%] Building C object CMakeFiles/child.dir/child.c.o
[100%] Linking C executable child
[100%] Built target child
$ cd build
```

Ввод из файла:

```
$ cat input.txt
out.txt
1 2 3
1.5 2.5
-1 0.25
100
$ ./parent < input.txt
$ cat out.txt
6.00
4.00
-0.75
100.00
109.25
```

Ввод через конвейер:

```
$ printf "res.txt\n10 20 30\n0.1 0.2 0.3\n-5 5\n" | ./parent
$ cat res.txt
60.00
0.60
0.00
60.60
```

Обработка ошибок (пустое имя файла, несуществующий каталог):

```
$ printf "\n1 2 3\n" | ./parent
parent: expected filename on first line
$ printf "nodir/out.txt\n1 2 3\n" | ./parent
child: cannot open file
parent: child reported error
```

Две пары процессов одновременно, объекты разделяемой памяти в /dev/shm:

```
$ (printf "a.txt\n1 1\n"; sleep 3; printf "2 2\n") | ./parent &
[1] 515
$ (printf "b.txt\n5 5\n"; sleep 3; printf "6 6\n") | ./parent &
[2] 519
$ ls -l /dev/shm
total 24
-rw------- 1 user user 4100 Nov  2 20:51 lab3_shm_515
-rw------- 1 user user 4100 Nov  2 20:51 lab3_shm_519
-rw------- 1 user user   32 Nov  2 20:51 sem.lab3_empty_515
-rw------- 1 user user   32 Nov  2 20:51 sem.lab3_empty_519
-rw------- 1 user user   32 Nov  2 20:51 sem.lab3_full_515
-rw------- 1 user user   32 Nov  2 20:51 sem.lab3_full_519
$ ps -o pid,ppid,cmd -C parent,child
    PID    PPID CMD
    515     315 ./parent
    517     515 child a.txt /lab3_shm_515 /lab3_empty_515 /lab3_full_515
    519     315 ./parent
    521     519 child b.txt /lab3_shm_519 /lab3_empty_519 /lab3_full_519
[1]-  Done
[2]+  Done
$ cat a.txt b.txt
2.00
4.00
6.00
10.00
12.00
22.00
$ ls /dev/shm
```

Пока программы работают, в /dev/shm видны объекты разделяемой памяти lab3_shm_<pid> размером 4100 байт (sizeof(shm_t)) и семафоры sem.lab3_empty_<pid>, sem.lab3_full_<pid>. У двух одновременно запущенных пар имена разные, поэтому они не мешают друг другу и записывают правильные результаты в свои файлы. Утилита ps показывает, что каждый процесс child – потомок своего parent и получил имена объектов в аргументах. После завершения программ каталог /dev/shm пуст: все объекты удалены.

Вывод

В ходе лабораторной работы я переделал первую лабораторную работу: обмен данными между родительским и дочерним процессами теперь идёт не через канал, а через именованную разделяемую память POSIX, отображённую в память обоих процессов функцией mmap. Я научился создавать объекты разделяемой памяти и именованные семафоры, задавать им уникальные имена и удалять их после работы.

Главное отличие от канала в том, что разделяемая память не синхронизирует процессы сама: чтение из канала блокировалось до появления данных, а общая память – это просто байты. Поэтому понадобились два семафора, которые задают строгую очередь «записал – прочитал». Зато данные не копируются через ядро, и такой способ обмена работает быстрее.

Основными сложностями были обработка данных, разделённых между порциями буфера, и ситуация, когда дочерний процесс завершается с ошибкой: без таймаута родитель навсегда ждал бы семафор. Первое решено хранением состояния разбора между порциями (проверено на вводе из 60 000 строк), второе – ожиданием семафора с таймаутом и проверкой дочернего процесса через waitpid.